Address Book

Problem 1: Create a Person Class

You are going to build an **Address Book** using **Object-Oriented Programming (OOP) in Python**.

In this problem, you will create a `Person` class. This class will be used in the next problem to build the complete address book. **Each entry in the address book will be an object (instance) of the `Person` class.**

Requirements

Define a class named `Person` with the following methods:

1. `__init__()` Method

The `__init__()` method should initialize the following information about a person:

- **Last Name**
- **First Name**
- **Phone Number**
- **Email Address(es)**

A person may have **one or multiple email addresses**.

2. `__str__()` Method

The `__str__()` method should return a properly formatted string containing the person's information.

The output should follow this format:

```
Last Name, First Name -- Phone Number: Phone Number -- Email Address(es): Email(s)
```

Example

```
Emily = Person(
    "Zhang",
    "Emily",
    "5559358150",
    "emzhang@mit.edu"
)

print(Emily)
```

Output:

```
Zhang, Emily -- Phone Number: 5559358150 -- Email Address(es):
emzhang@mit.edu
```

The class should also support **multiple email addresses**:

```
Wesley = Person(
    "Graybill",
    "Wesley",
    "5551940325",
    ["wgray496@mit.edu", "wdgraybill@gmail.com"]
)

print(Wesley)
```

Output:

```
Graybill, Wesley -- Phone Number: 5551940325 -- Email Address(es):
wgray496@mit.edu, wdgraybill@gmail.com
```

Task

Write the `Person` class by implementing:

- `__init__()`
- `__str__()`

Then create at least **two `Person` objects** and demonstrate their output using `print()`

Note: You may refer to Python documentation to understand how the special methods `__init__()` and `__str__()` work.

Address Book

Problem 2: Create an AddressBook Class

In this problem, you will use the **Person class from Problem 1** to create an `AddressBook` class for storing the contacts of your friends and family.

The address book should allow you to:

1. Add a new contact.
2. Search for contacts by **last name**.
3. Optionally search by **first name** when multiple contacts have the same last name.

Each contact stored in the address book should be an **instance of the `Person` class** created in Problem 1.

Requirements

Define a class named `AddressBook` with the following methods:

1. `__init__()` Method

Create an `__init__()` method that initializes an empty address book.

You should use Python's built-in **dictionary (`dict`)** data type to store the contacts.

2. `add_contact()` Method

Create a method named `add_contact()` that accepts a `Person` object and adds it to the address book.

For example:

```
person = Person(
    "Kamil",
    "Ali",
    "5551234567",
    "ali@example.com"
)

address_book.add_contact(person)
```

3. `lookup_contact()` Method

Create a method named `lookup_contact()` that searches for contacts by **last name**.

The method should:

- Accept the **last name** as a required argument.
- Print every contact matching that last name.
- Print each matching contact on a **new line**.

Example:

```
address_book.lookup_contact("Kamil")
```

Output:

```
Kamil, Ali -- Phone Number: 5551234567 -- Email Address(es): ali@example.com
Kamil, Sama -- Phone Number: 5559876543 -- Email Address(es): sama@example.com
```

⭐ Additional Challenge: Search by First Name

Extend the `lookup_contact()` method so that the user can **optionally provide a first name**.

If only the last name is provided, display **all contacts** with that last name.

If both the last name and first name are provided, display only the contact matching **both names**.

You can use a **default argument** to make the first name optional.

Example 1 — Search by Last Name

```
address_book.lookup_contact("Kamil")
```

Output:

```
Kamil, Ali -- Phone Number: 5551234567 -- Email Address(es): ali@example.com
Kamil, Sama -- Phone Number: 5559876543 -- Email Address(es): sama@example.com
```

Example 2 — Search by Last Name and First Name

```
address_book.lookup_contact("Kamil", "Ali")
```

Output:

```
Kamil, Ali -- Phone Number: 5551234567 -- Email Address(es): ali@example.com
```

Expected Class Structure

Your `AddressBook` class should contain:

```
AddressBook
│
├── __init__()
```

```
├── add_contact()
└── lookup_contact()
```

Task

Create the `AddressBook` class using the `Person` class from **Problem 1**.

Your program should demonstrate that you can:

- Create an empty address book.
- Create multiple `Person` objects.
- Add contacts to the address book.
- Search contacts using their last name.
- Optionally narrow the search using their first name.

Hint: To complete the optional first-name search, review **default arguments / optional arguments in Python**.